

TD7 – éléments de corrigé (exercice 6)

Structure Classe

```
{  
    champ _code : chaîne de caractères  
    champ _nom : chaîne de caractères  
    champ _prix : réel  
  
    Constructeur : Classe  
    Entrée :  
        - code : chaîne de caractères  
        - nom : chaîne de caractères  
        - prix : réel  
  
    Début  
    |  
    |     _code = code  
    |     _nom = nom  
    |     _prix = prix  
    Fin  
}
```

Structure Passager

```
{  
    champ _numero : entier  
    champ _nom : chaîne de caractères  
    champ _prenom : chaîne de caractères  
    champ _codeClasse : chaîne de caractères  
  
    Constructeur : Passager  
    Entrée :  
        - numero : entier  
        - nom : chaîne de caractères  
        - prenom : chaîne de caractères  
        - codeClasse : chaîne de caractères  
  
    Début  
    |  
    |     _numero = numero  
    |     _nom = nom  
    |     _prenom = prenom  
    |     _codeClasse = codeClasse  
    Fin  
}
```

Structure Bagage

```
{  
    champ _numero : entier  
    champ _poids : entier  
    champ _numBillet : entier  
  
    Construction : Bagage  
    Entrée :  
        - numero : entier  
        - poids : entier  
        - numBillet : entier  
  
    Début  
    |  
    |     _numero = numero  
    |     _poids = poids  
    |     _numBillet = numBillet  
    Fin  
}
```

Fonction : **ChargerListeClasses**

Entrée : - chemin : chaîne de caractères

Variables : - fs : flux de lecture
- classes : Liste<structure Classe>
- line : chaîne de caractères
- elts : tableau 1D de chaînes de caractères

Sortie : Liste<structure Classe>

Début

```
classes = ListeVide()
Si Existe(chemin) Alors
    Ouvrir Fichier fs depuis chemin en mode Lecture
    Tant Que (line = fs.LireLigne()) ≠ EOF Faire
        elts = line.Separer(";")
        classes.Ajouter(Classe(elts[0], elts[1], Réel(elts[2])))
    Fin Tant Que
    Fermer Fichier fs
Fin Si

Retourner classes
```

Fin

Fonction : **ChargerListePassagers**

Entrée : - chemin : chaîne de caractères

Variables : - fs : flux de lecture
- passagers : Liste<structure Passager>
- line : chaîne de caractères
- elts : tableau 1D de chaînes de caractères

Sortie : Liste<structure Passager>

Début

```
passagers = ListeVide()
Si Existe(chemin) Alors
    Ouvrir Fichier fs depuis chemin en mode Lecture
    Tant Que (line = fs.LireLigne()) ≠ EOF Faire
        elts = line.Separer(";")
        passagers.Ajouter(Passager(Entier(elts[0]), elts[1], elts[2], elts[3]))
    Fin Tant Que
    Fermer Fichier fs
Fin Si

Retourner passagers
```

Fin

Fonction : **ChargerListeBagages**

Entrée : - chemin : chaîne de caractères

Variables : - fs : flux de lecture
- bagages : Liste<structure Bagage>
- line : chaîne de caractères
- elts : tableau 1D de chaînes de caractères

Sortie : Liste<structure Bagage>

Début

```
bagages = ListeVide()
Si Existe(chemin) Alors
    Ouvrir Fichier fs depuis chemin en mode Lecture
    Tant Que (line = fs.LireLigne()) ≠ EOF Faire
        elts = line.Separer(";")
        bagages.Ajouter(Bagage(Entier(elts[0]), Entier(elts[1]), Entier(elts[2])))
    Fin Tant Que
    Fermer Fichier fs
Fin Si

Retourner bagages
```

Fin

Fonction : ChercherNomClasse

Entrée :
- classes : Liste<structure Classe>
- code : chaîne de caractères

Variables :
- c : structure Classe

Sortie :
chaîne de caractères

Début

```
Pour Chaque c dans classes Faire
|
|   Si c._code == code Alors
|       Retourner c._nom
|   Fin Si
Fin Pour Chaque
```

```
Retourner "" // erreur
```

Fin

Fonction : ChercherBagagesPassager

Entrée :
- bagages : Liste<structure Bagage>
- numero : entier

Variables :
- b : structure Bagage
- bagagesPassager : Liste<structure Bagage>

Sortie :
Liste<structure Bagage>

Début

```
bagagesPassager = ListeVide()
Pour Chaque b dans bagages Faire
|
|   Si b._numBillet == numero Alors
|       bagagesPassager.Ajouter(b)
|   Fin Si
Fin Pour Chaque
```

```
Retourner bagagesPassager
```

Fin

Procédure : AfficherPassagers

Entrée :
- passagers : Liste<structure Passager>
- classes : Liste<structure Classe>
- bagages : Liste<structure Bagage>

Variables :
- poids : entier
- bagagesDuPassager : Liste<structure Bagage>
- p : structure Passager
- nomClasse : chaîne de caractères
- b : structure Bagage

Sortie :
/

Début

```
Afficher("Numéro Passager Classe Nombre bagages Poids cumulé")
Afficher("-----")

Pour Chaque p dans passagers Faire
|
|   nomClasse = ChercherNomClasse(classes, p._codeClasse)
|   bagagesDuPassager = ChercherBagagesPassager(bagages, p._numero)
|   poids = 0
|   Pour Chaque b dans bagagesDuPassager Faire
|       poids = poids + b._poids
|   Fin Pour Chaque
|   Afficher(p._numero + " " + p._prenom + " " + p._nom + " " + nomClasse + " ")
|   Afficher(bagagesDuPassager.Taille() + " " + poids)
Fin Pour Chaque
```

Fin

Fonction : **ChercherPrixClasse**

Entrée :
- classes : Liste<structure Classe>
- code : chaîne de caractères

Variables :
- c : structure Classe

Sortie :
réel

Début

```
Pour Chaque c dans classes Faire
|
|   Si c._code == code Alors
|       Retourner c._prix
|   Fin Si
Fin Pour Chaque
```

```
Retourner 0.0 // erreur
```

Fin

Procédure : **CalculerPrixBilletPassager**

Entrée :
- bagages : Liste<structure Bagage>
- prixBillet : réel

Variables :
- prix : réel
- poids : entier
- b : structure Bagage

Sortie :
réel

Début

```
prix = prixBillet
poids = 0.0
```

```
Pour Chaque b dans bagages Faire
|   poids = poids + b._poids
Fin Pour Chaque
```

```
Si poids > 20 ET poids <= 30 Alors
|   prix = prix * 1.10
Sinon Si poids > 30 ET poids <= 40 Alors
|   prix = prix * 1.20
Sinon Si poids > 40 Alors
|   prix = prix * 1.30
Fin Si
```

```
Retourner prix
```

Fin

Procédure : **RechercherPassager**

Entrée :
- passagers : Liste<structure Passager>
- num : entier
- référence p : structure Passager

Variables :
- pas : structure Passager

Sortie :
booléen

Début

```
Pour Chaque pas dans passager Faire
|   Si pas._numero == num Alors
|       p = pas
|       Retourner VRAI // vrai, p contient le bon passager
|   Fin Si
Fin Pour Chaque
```

```
Retourner FAUX // faux, p n'a pas été modifié
```

Fin

Fonction : **CalculerChiffreAffairesVol**

Entrée :
- passagers : Liste<structure Passager>
- classes : Liste<structure Classe>
- bagages : Liste<structure Bagage>

Variables :
- ca : réel
- p : structure Passager
- prixBase : réel
- bagagesP : Liste<structure Bagage>

Sortie :
réel

Début

ca = 0.0

Pour Chaque p **dans** passagers **Faire**

 prixBase = ChercherPrixClasse(classes, p._codeClasse) // prix du billet dans la surtaxe

 bagagesP = ChercherBagagesPassager(bagages, p._numero)

 ca = ca + CalculerPrixBilletPassager(bagagesP, prixBase) // prix avec la surtaxe

Fin Pour Chaque

Retourner ca

Fin

Algorithme : **Main**

Variables :

- numVol, choix, numPassager : entiers
- classes : Liste<structure Classe>
- passagers : Liste<structure Passager>
- bagages : Liste<structure Bagage>
- trouve, menu : booléens
- prixBase, prix1, ca : réels
- bagages1 : Liste<structure Bagage>
- p1 : structure Passager

Début

```
Afficher("Saisir un numéro de vol : ")
numVol = SaisirEntier()

classes = ChargerListeClasses("C" + numVol + ".txt")
passagers = CharcherListePassagers("P" + numVol + ".txt")
bagages = ChargerListeBagages("B" + numVol + ".txt")

menu = VRAI
Tant Que menu Faire
    Afficher("1 - Prix du billet d'un passager")
    Afficher("2 - Liste des passagers")
    Afficher("3 - Chiffre d'affaires")
    Afficher("4 - Quitter")
    Afficher("Saisir un choix : ")
    choix = SaisirEntier()

    Selon Choix choix
        choix 1
            Afficher("Saisir un numéro de passager : ")
            numPassager = SaisirEntier()
            trouve = RechercherPassager(passagers, numPassager, référence p1)
            Si trouve Alors
                bagages1 = ChercherBagagesPassager(bagages, numPassager)
                prixBase1 = CalculerPrixClasse(classes, p1._codeClasse)
                prix1 = CalculerPrixBilletPassager(bagages1, prixBase1)
                Afficher("Prix du billet : " + prix1 + " €")
            Fin Si
            Quitter
        choix 2
            AfficherPassagers(passagers, classes, bagages)
            Quitter
        choix 3
            ca = CalculerChiffreAffairesVol(passagers, classes, bagages)
            Afficher("Chiffre d'affaire du vol : " + ca + " €")
            Quitter
        choix par défaut
            menu = FAUX
            Quitter
    Fin Selon Choix
Fin Tant Que
```

Fin